The original manuscript, "Budgeted Classification with Rejection: An Evolutionary Method with Multiple Objectives", is written to be self-contained. However, in this document, we include several experiments and demonstrations in case they are informative for readers or aid reproducibility. If published, the repository will be de-anonymized.

1 Symbols and Notation

Symbol	Description
β	stage count bias parameter
b	elitism proportion
$\mathcal{C}$	feature costs
$\mathscr{C}(Q_j, \mathbf{x})$	returns class probabilities for $\mathbf{x}$ after j^{th} stage evaluation
E_0^*	unique non-dominated solutions in a given generation
ϵ_{EA}	95% margin of error
$\mathscr{E}(g_1, g_2, g_3)$	Euclidean norm of objectives
$\mathscr{E}^*(g_1, g_2, g_3^*)$	Euclidean norm of objectives with a "global" measure of inverse cost
$\mathcal{F}$	feature set
G_h	refers to the hth generation
$\|G\|$	population size
g	convergence parameter
k	max number of stages
M	elite population size (not fixed)
$\hat{m}$	mutation rate
n	number of features
N	number of test/validation records
$\mathscr{N}(G_h)$	returns the non-dominated front in generation G_h
$\mathcal{P}_{(n,j)}$	ordered j-partitions of $\mathcal{F}$
$\bar{p_j}$	maximum class probability after j^{th} stage
$\hat{p}$	confidence threshold
$\|Q\|$	stage count of solution Q
Q_m	final stage/features evaluated for a specific input
Q_j	features in stage j
$[Q]$	chromosome representation of solution Q
$[Q]_i$	stage assignment for feature i
$[\bar{Q}]_{max}$	Element-wise mean vector of top-scoring chromosomes
$\hat{r}$	recombination rate
r_{max}	rank of the non-dominated solutions in a specified population
$\mathcal{S}_{(n,k)}$	all solutions with at most k stages
T_i	cost class of ith feature
$\bar{X}_h$	(sample) mean number of globally optimal solutions present in G_h
$\lceil \cdot \rceil$	returns next smallest integer
$\mathbb{1}[\cdot]$	indicator function
$\succ$	Pareto dominance relation

Table 1: Notation and Symbols used in Manuscript

2 Feature Descriptions

Feature	Cost	Description
$\mathcal{F}_1$	1	Age of Patient
$\mathcal{F}_2$	2	Anaemia: {0,1}
$\mathcal{F}_3$	2	High Blood Pressure: {0,1}
$\mathcal{F}_4$	2	Creatinine Phosphokinase Level
$\mathcal{F}_5$	2	Diabetes: {0,1}
$\mathcal{F}_6$	2	Ejection Fraction
$\mathcal{F}_7$	2	Platelets
$\mathcal{F}_8$	1	Sex: {0,1}
$\mathcal{F}_9$	2	Serum Creatinine
$\mathcal{F}_{10}$	2	Serum Sodium
$\mathcal{F}_{11}$	1	Smoking: {0,1}
$\mathcal{F}_{12}$	1	Follow-Up Period
Target	N/A	Death Event: {0,1}

Heart

Feature	Cost	Description
$\mathcal{F}_1$	1	Number of times pregnant
$\mathcal{F}_2$	3	Plasma glucose concentration
$\mathcal{F}_3$	2	Diastolic blood pressure
$\mathcal{F}_4$	2	Triceps skin fold thickness
$\mathcal{F}_5$	3	2-Hour serum insulin
$\mathcal{F}_6$	2	Body mass index
$\mathcal{F}_7$	2	Diabetes pedigree function
$\mathcal{F}_8$	1	Age
Target	N/A	Diabetes diagnosis: {0,1}

Pima

Feature	Cost	Description
$\mathcal{F}_1$	2	Categorical: {0,1}
$\mathcal{F}_2$	2	Continuous
$\mathcal{F}_3$	2	Continuous
$\mathcal{F}_4$	1	Categorical: {1,2,3}
$\mathcal{F}_5$	2	Categorical: {1...14}
$\mathcal{F}_6$	1	Categorical: {1...9}
$\mathcal{F}_7$	2	Continuous
$\mathcal{F}_8$	3	Categorical: {0,1}
$\mathcal{F}_9$	1	Categorical: {0,1}
$\mathcal{F}_{10}$	2	Continuous
$\mathcal{F}_{11}$	1	Categorical: {0,1}
$\mathcal{F}_{12}$	1	Categorical: {1,2,3}
$\mathcal{F}_{13}$	1	Continuous
$\mathcal{F}_{14}$	2	Continuous
Target	N/A	Class {0,1}

Credit

Synthetic15 uses arbitrary features, so no feature description table is displayed for either.

3 Operator Pseudocode

Algorithm 1: Recombination

$[P_A]$, $[P_B]$ are parents selected for recombination in G_h
Initiate child $[C] \leftarrow [0, 0, \ldots, 0]$
$|C| \leftarrow rand\left\{\left\lfloor \frac{|P_A|+|P_B|}{2} \right\rfloor, |P_A|, |P_B|\right\}$;
// $0 \leq i < n$
for each ith feature **do**
  $R = rand\{P_A, P_B\}$;
  Assign stage to the ith feature of the child
  $[C]_i \leftarrow_{round} \left(\frac{[R]_i+1}{|R|}\right)|C|$;
  $[C]_i \leftarrow [C]_i - 1$;
  $[C]_i \leftarrow \max([C]_i, 0)$;
end
return $[C]$

Algorithm 2: Mutation

Q is a new chromosome
// $0 \leq i < n$
for each ith feature **do**
  if $(rand([0,1]) \leq \hat{m}$ & $count([Q]_i, [Q]) > 1)$ **then**
    $[Q]_i \sim BB(\min(|\mathrm{Q}|, k-1), 1, \beta)$;
  end
end

The condition $count([Q]_i, [Q]) > 1$ checks if there is more than one occurrence of $[Q]_i$ in $[Q]$ and is considered to ensure that mutation will not result in an empty stage. The draw from $\min(|\mathrm{Q}|, k-1)$ elements ensures that the allowed number of stages k is not exceeded.

Algorithm 3: `EMSCO`

```
G_0 ← init();
|G|_cpy ← |G|;
h ← 0;
while not converged() do
    rank(G_h);
    sort(G_h, key = f(·));
    elite_size ← max(round(b · |G*_h|), |E*_0|);
    elite_pop ← G*_h [0 : elite_size];
    G_{h+1} ←_append elite_pop;
    if elite_size = |G| then
        |G| += inc;
    end
    if elite_size < (|G| − inc) and |G| > |G|_cpy then
        |G| = |G| − inc;
    end
    while |G_{h+1}| < |G| do
        P_A, P_B ← select();
        [C] ← recombine([P_A], [P_B]);
        [C] ← mutate([C]);
        if gaps([C]) then
            compress([C]);
        end
        G_h ←_append C;
    end
    h ← h + 1;
end
return N(G_h); // return non-dominated solutions in G_h
```

Note, due to its elitism protocol, `EMSCO` requires $|G| - M$ chromosomes be created/evaluated at each generation, where,

$$M = elite_size = \max\left(round\left(b \cdot |G_h^*|\right), |E_0^*|\right).$$

4 Parameter Tuning

`EMSCO`'s parameters are tuned independently for each data set on training/validation splits with a simple grid/sweep method [1]. Population size ($|G|$), mutation rate ($\hat{m}$), recombination rate ($\hat{r}$), elite percent threshold (b), and the mutation bias parameter (β) are subject to tuning. Continuous parameters ($\hat{m}, \hat{r}, b, \beta$) are discretized within an appropriate range, and all possible combinations of parameters in these ranges are tested. To evaluate performance of a particular combination of parameters, `EMSCO` is run ten times on the validation set using the given parameters with convergence settings fixed ($g = 50$ and $max_iter = 100$) and $k = \max(\lceil \frac{n}{2} \rceil, 10)$. The top scoring solution (according to $f()$) in each of the ten terminal generations is recorded and these are used to compute average element-wise performance for a given parameter set according to accuracy, coverage, and cost. The average performance for each parameter set is then sorted using a modified cost objective that is population-independent :$1 - \frac{g_3^*}{\sum_{i=1}^{n} \mathcal{C}_i}$. We pick the parameter combination that maximizes

$$\mathscr{E}^* = \sqrt{g_1^2 + g_2^2 + (1 - \frac{g_3^*}{\sum \mathcal{C}_i})^2}$$

. To mitigate computational expense incurred by large population sizes, if $|\mathscr{E}^*(Q_1) - \mathscr{E}^*(Q_2)| < d$, the parameter combination with lower population size is chosen. We used $d = .1$.

Parameter	Tested Values
$\hat{m}$	$[0.025, 0.1]_{0.025}$
$\hat{r}$	$[0.70, 0.90]_{0.1}$
β	$[2.0, 3.0]_{0.5}$
b	$[0.1, 0.3]_{0.1}$
$\lvert G \rvert$	$\{200, \ldots, 400\}_{50}$

Table 2: Parameter Tuning Domain EMSCO

See Table 2 for the tested parameter ranges. The subscript in each entry is used to denote the size of gaps between elements. These ranges and increments are chosen to encompass a suitable breadth of parameter choices under the constraint of maintaining computational tractability—in our setup, tuning is an expensive procedure. Up to 100 generations must be created and evaluated for each parameter combination 10 times. This tuning procedure is not exhaustive, but more rigorous tuning should only improve performance of `EMSCO` in experiments. Note, confidence threshold $\hat{p}$ is not subject to tuning as the method assumes that this threshold is set *a priori*.

GM and $CSTC$ are tuned using a simple grid search procedure. See [2] and [3] for a description of the parameters. Once we have g_1, g_2, and $\frac{g_3^*}{\sum_{i=1}^{n} \mathcal{C}_i}$, we sort each parameter choice according to $\mathscr{E}^*$. Note, $g_1 = 1$ for all possible parameter choices, since these methods are not selective (i.e., do not apply a terminal reject option). The same selection procedure is then applied as in EMSCO, however, there is no population size "tie-breaker". GM's λ parameter was tested in range $(0, 0.1, \ldots, 10)$. CSTC's λ parameter was tested in this same range. CaCT LASSO uses a nearly identical tuning procedure as GM and $CSTC$ to find a λ (ℓ_1-regularization coefficient) providing the best multi-objective performance on the validation set.

5 Optimization Behavior

5.1 EA Benchmark

To establish a point of reference with *general-purpose* multi-objective evolutionary algorithms, we compare per-generation optimization performance of `EMSCO` to NSGA-II [4] with random reset mutation that is conformable to our chromosome representation. The initial population of NSGA-II is generated with the same $init()$ procedure used by `EMSCO`, and the same ℓ_2-regularized logistic regression classifiers used by `EMSCO` are used. The performance in this experiment is measured during optimization, i.e., on the training/validation sets, *not* the test set (as is used in the main paper to chart performance in Table II)

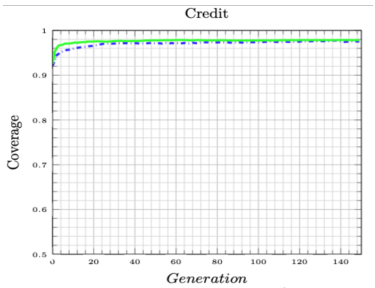
Parameters for NSGA-II are tuned in the same manner as for `EMSCO` but with a wider range of mutation and crossover rates and no mutation bias parameter (β). Similarly for `EMSCO`, NSGA-II, performance at each generation is determined by the chromosome with greatest $\mathscr{E}(\cdot)$. `EMSCO` and NSGA-II are run 50 times and their average performance on the training set recorded with respect to the 50 samples. These 50 samples are not always sufficient to generate smooth curves in each plot for NSGA-II (e.g., Pima), but time and computational constraints prevent us from including more runs in these preliminary investigations.

Note that `EMSCO` uses substantially fewer function evaluations as each population requires at most $(|G| - M)$ new chromosomes to be evaluated, whereas NSGA-II generates $|G|$ solutions at each generation. Additionally, `EMSCO`'s halting conditions can prevent additional function evaluations during each run. We set $inc = 0$ and $g = 50$.

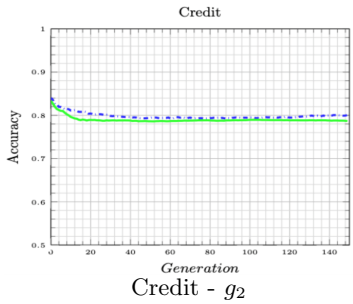
In the following plots `EMSCO`'s performance is in green, and NSGA-II's performance is in blue. Generally, `EMSCO` exhibits less variance in per-generation performance and is quicker to arrive at good solutions. While maintaining near-equal accuracy, `EMSCO` consistently achieves higher coverage and lower cost than NSGA-II. In certain settings, NSGA-II's slower convergence may be advantageous for maintaining diversity, but this set of instances suggests that `EMSCO`'s faster convergence is not detrimental to end-performance.

Parameter	Tested Values
$\hat{m}$	$[0.025, 0.2]_{0.025}$
$\hat{r}$	$[0.70, 0.95]_{0.05}$
$\lvert G \rvert$	$\{200, \ldots, 400\}_{50}$
Tournament Size	$\{10, 20, 30, 40, 50, 100\}$

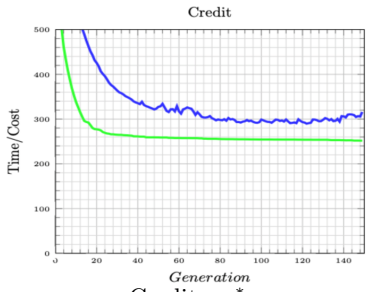
Table 3: Parameter Tuning Domain NSGA-II



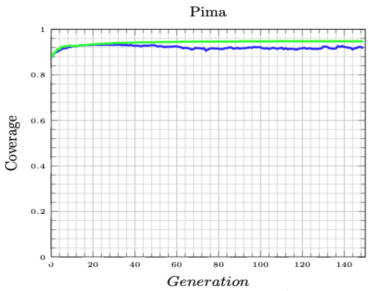
Credit - g_1



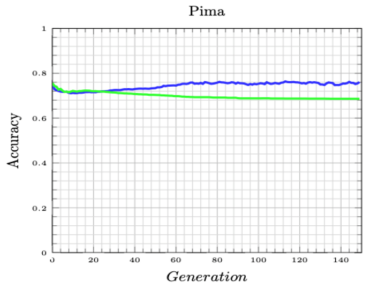
Credit - g_2



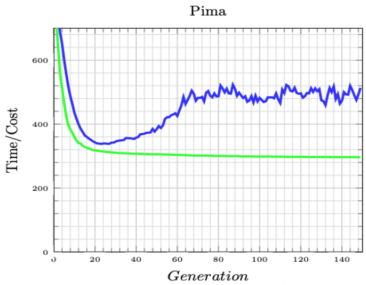
Credit - g_3^*



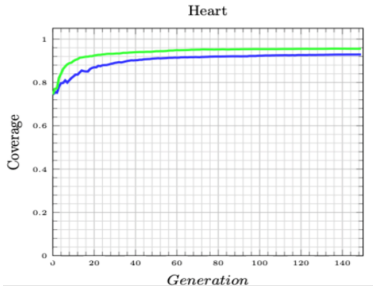
Pima - g_1



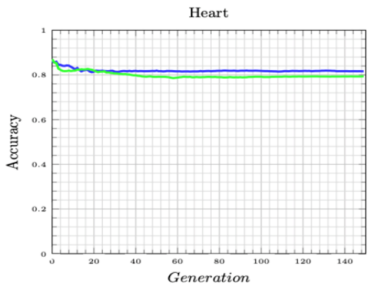
Pima - g_2



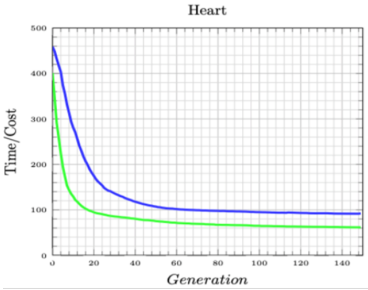
Pima - g_3^*



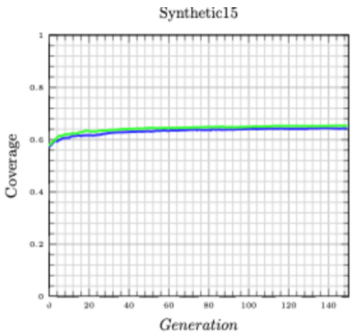
Heart - g_1



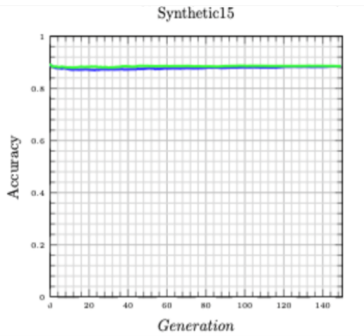
Heart - g_2



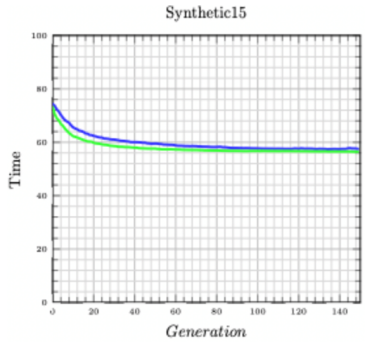
Heart - g_3^*



Synthetic15 - g_1



Synthetic15 - g_2



Synthetic15 - g_3^*

6 EMSCO Performance/Behaviour Demo

See `heart_demo.txt`. This brief demonstration shows EMSCO's iterative, population-based optimization behaviour and tracks the number of verified globally optimal solutions present in EMSCO's elite population at each generation.

The default EMSCO parameters suggested in the main paper are used ($\hat{m} = \frac{1}{n}$, $\hat{r} = 0.8$, $|G| = 300$, $b = 0.2$, and $\beta = 2$). As in the main paper's global experiment, $k = 4$, and $\hat{p} = 0.75$.

References

[1] K. De Jong, "Parameter setting in eas: a 30 year perspective," in *Parameter setting in evolutionary algorithms*. Springer, 2007, pp. 1–18.

[2] Z. Xu, K. Weinberger, and O. Chapelle, "The greedy miser: Learning under test-time budgets," 2012.

[3] Z. E. Xu, M. J. Kusner, K. Q. Weinberger, M. Chen, and O. Chapelle, "Classifier cascades and trees for minimizing feature evaluation cost," *Journal of Machine Learning Research*, vol. 15, no. 62, pp. 2113–2144, 2014. [Online]. Available: http://jmlr.org/papers/v15/xu14a.html

[4] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.